Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине: «КиСБ»
Тема: «Исследование алгоритмов генерации псевдослучайных последовательностей и методов тестирования чисел на простоту»

Выполнил:
Студент 4-го курса
Группы АС-66
Цеван К. А.
Проверил:
Хацкевич А. С.

Брест 2026

Цель работы

Освоить программную реализацию генератора псевдослучайных последовательностей, исследовать его период и статистические свойства, реализовать генерацию кандидатов и проверку чисел на простоту.

Ход работы

Вариант 12

1. Генерация и тестирование псевдослучайных последовательностей

По варианту требуется построить LFSR-генератор для трёх многочленов: $f_1(x) = x^5 + x^2 + 1$; $f_2(x) = x^5 + 1$; $f_3(x) = x^5 + x^4 + x^2 + 1$. Для каждого многочлена определить примитивность по модулю 2, подобрать начальное состояние, исследовать период и распределение выходной последовательности.

Используется пятиразрядный регистр. Начальное состояние — 00001. На каждом шаге выдаётся младший бит, регистр сдвигается вправо, а в старший разряд записывается XOR выбранных разрядов. Маски обратной связи для f_1, f_2, f_3 равны соответственно 00101, 00001 и 10101.

Если выходные биты обозначены b_t, то выполняются рекуррентные соотношения $b_{t+5} = b_{t+2}$ XOR b_t; $b_{t+5} = b_t$; $b_{t+5} = b_{t+4}$ XOR b_{t+2} XOR b_t. Нулевое начальное состояние не используется: регистр из него не выходит.

Максимальный период пятиразрядного LFSR равен $2^5 - 1 = 31$. Проверены все 31 ненулевых начальных состояния. Только для f_1 каждое состояние входит в цикл длины 31. Поэтому f_1 — примитивный многочлен; f_2 и f_3 не являются примитивными. При выбранном начальном состоянии их периоды равны 5 и 15.

Многочлен	T при x_0=1	Примитивность
x^5+x^2+1	31	Да
x^5+1	5	Нет
x^5+x^4+x^2+1	15	Нет

1.1 Программная реализация генератора

Генератор хранит только состояние регистра и маску обратной связи. Последовательность обрабатывается по одному биту; массив всей последовательности в памяти не создаётся.

```
struct Lfsr {
    unsigned state, taps;
    Lfsr(unsigned mask, unsigned seed = 1) : state(seed), taps(mask) {}
    unsigned next() {
        unsigned value = state & taps;
        unsigned feedback = 0;
        for (unsigned i = 0; i < 5; ++i)
            feedback ^= (value >> i) & 1u;
        unsigned bit = state & 1u;
        state = (state >> 1) | (feedback << 4);
        return bit;
    }
};

unsigned period(unsigned mask, unsigned seed) {
    Lfsr gen(mask, seed);
    for (unsigned t = 1; t <= 31; ++t) {
        gen.next();
        if (gen.state == seed) return t;
    }
    return 0;
}

bool primitive(unsigned mask) {
    bool result = true;
    for (unsigned seed = 1; seed < 32; ++seed)
        result = result & (period(mask, seed) == 31);
    return result;
}
```

Методика статистического исследования

Для каждого многочлена сформировано L = 10 000 битов. Подсчитаны частоты n_0 и n_1, выборочное среднее $M = n_1/L$ и дисперсия $D = M(1 - M)$. Для равномерного распределения ожидаются M = 0,5 и D = 0,25.

Частотная гипотеза H_0: вероятности нуля и единицы равны 0,5. Статистика Пирсона $\chi^2 = (n_0 - L/2)^2/(L/2) + (n_1 - L/2)^2/(L/2)$. При одной степени свободы и уровне значимости 0,05 критическое значение равно 3,841.

Дополнительно рассчитана автокорреляция для лагов 1–20. Частотный критерий рассматривается как учебная диагностика: периодические биты LFSR не являются независимыми случайными наблюдениями.

1.2 Результаты исследования

f(x)	n_0	n_1	M	D	χ^2
f1	4839	5161	0,5161	0,2497	10,368
f2	8000	2000	0,2000	0,1600	3600,000
f3	5333	4667	0,4667	0,2489	44,356

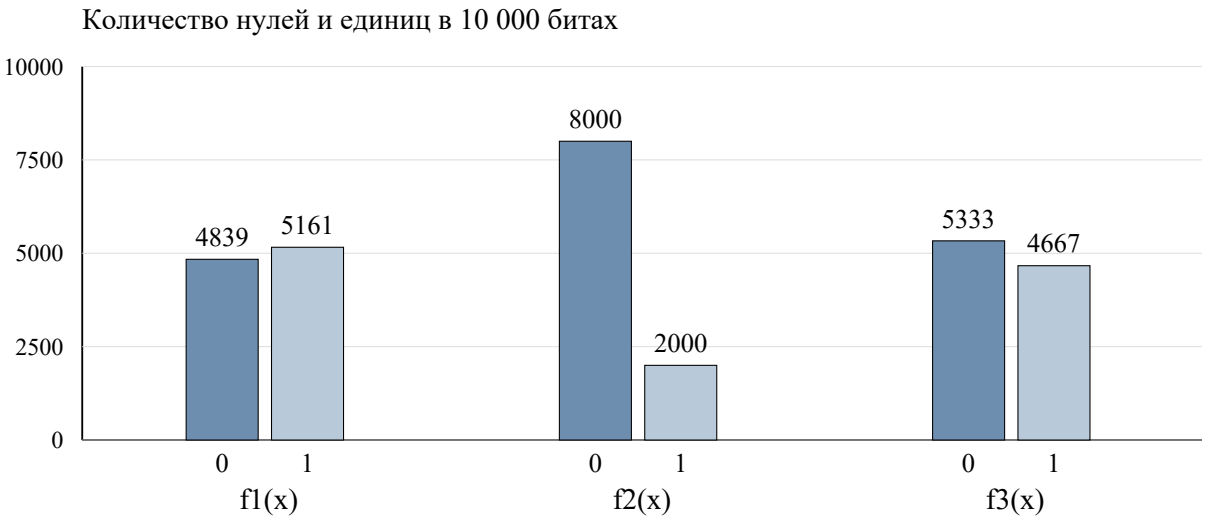


Рисунок 1 — Частоты нуля и единицы для трёх многочленов.

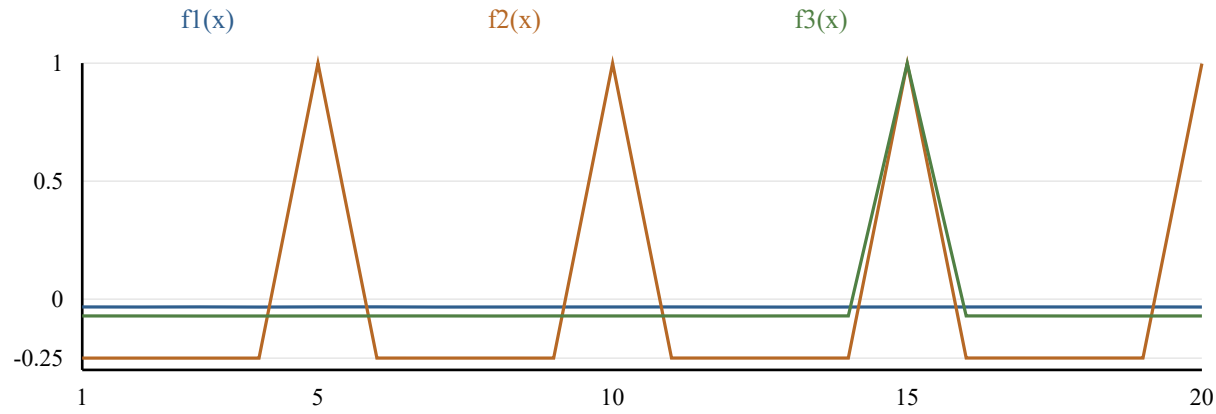


Рисунок 2 — Автокорреляция при лагах от 1 до 20.

Для всех трёх наборов на выборке 10 000 битов χ^2 превышает порог 3,841. Наиболее близкое к равномерному распределение даёт f_1, но и у него на полном периоде 16 единиц и 15 нулей. При многократном повторении периода это небольшое различие накапливается. У f_2 повторение видно через каждые 5 битов, у f_3 — через 15. Из трёх вариантов для следующего этапа выбран f_1 с периодом 31.

2. Тестирование на простоту и генерация простых чисел

По варианту 12 используется тест Рабина—Миллера. Из выходов LFSR с f_1 формируется p-битовое число n. Старший и младший биты устанавливаются в 1, что обеспечивает нужную разрядность и нечётность.

```
U64 candidate(Lfsr& gen, unsigned bits) {
    U64 n = 0;
    for (unsigned i = 0; i < bits; ++i) n |= U64(gen.next()) << i;
    return n | (U64(1) << (bits - 1)) | 1;
}
```

Кандидат проверяется делением на все 54 простых числа меньше 256, от 2 до 251. Если кандидат сам равен малому простому, он не отбрасывается. Затем выполняются пять раундов Рабина—Миллера. После неудачи проверяется n + 2; такой последовательный перебор допускается на странице 54 методички. При выходе за разрядность поиск продолжается с минимального нечётного числа той же разрядности.

Для нечётного n число $n - 1$ представляется как $2^s d$, где d нечётно. Для выбранного основания a вычисляется $x = a^d \bmod n$. Раунд пройден, если $x = 1$, $x = n - 1$ либо одно из следующих $s - 1$ возведений x в квадрат по модулю n даёт $n - 1$.

Основания равномерно и независимо выбираются из диапазона $2 \ldots n - 2$ с помощью системного источника случайных чисел. Это нужно для вероятностной оценки теста; короткий LFSR используется для кандидата. Для фиксированного составного кандидата вероятность пройти пять независимых раундов не превышает $4^{-5} = 1/1024$. Результат теста — «вероятно простое», а не доказательство простоты.

```
bool miller_round(U64 n, U64 a) {
    U64 d = n - 1;
    unsigned s = 0;
    for (unsigned i = 0; i < 64; ++i) {
        unsigned take = unsigned((d & 1) == 0);
        d >>= take;
        s += take;
    }
    U64 x = power_mod(a % n, d, n);
    bool passed = (x == 1 || x == n - 1);
    for (unsigned r = 1; r < 64; ++r) {
        x = multiply_mod(x, x, n);
        passed = passed | ((r < s) & (x == n - 1));
    }
    return passed;
}
```

2.1 Результаты генерации простых чисел

Для разрядностей 16, 32 и 64 получено по три результата. Время измерялось от начала перебора кандидатов до окончания проверки найденного числа. В таблице указаны общее количество кандидатов, количество отсечённых ситом и количество отклонённых тестом Рабина—Миллера.

Бит	Полученное число n	Всего	Сито	Тест	Время, мс
16	52517	3	2	0	2,259
16	44753	6	5	0	0,654
16	59029	3	2	0	0,591
32	2932329761	1	0	0	0,600
32	3613648553	13	8	4	3,396
32	2880566089	1	0	0	0,844
64	15520502249930870051	2	1	0	0,620
64	10797654590123799373	3	1	1	1,211
64	9616942675172031733	18	13	4	2,412

Для каждой строки выполняется равенство: всего кандидатов = отсеяно ситом + отклонено тестом + 1. Присутствие старшего бита проверено для каждой заданной разрядности. Время и случайные основания могут изменяться при повторном запуске.

Ключевые фрагменты модульной арифметики

```
U64 add_mod(U64 a, U64 b, U64 n) {
    U64 mask = U64(0) - U64(a >= n - b);
    return ((a - (n - b)) & mask) | ((a + b) & ~mask);
}

U64 multiply_mod(U64 a, U64 b, U64 n) {
    U64 result = 0;
    for (unsigned i = 0; i < 64; ++i) {
        U64 sum = add_mod(result, a, n);
        U64 mask = U64(0) - ((b >> i) & 1);
        result = (sum & mask) | (result & ~mask);
        a = add_mod(a, a, n);
    }
    return result;
}
```

Умножение и возведение в степень выполняются без переполнения 64-разрядного типа. В арифметических функциях используется фиксированное число шагов и выбор результата битовой маской. Сам поиск простого числа имеет переменное число кандидатов, как и схема в методичке.

2.2 Проверка программы и запуск

Проверка LFSR подтвердила периоды 31, 5 и 15. Для теста простоты выполнено сравнение с точным пробным делением на нечётных числах меньше 2000; отдельно проверены малые простые, составные числа, числа Кармайкла 561, 1105 и 1729, а также большие значения в пределах 64 бит. Проверены разрядность кандидатов и работа просеивания.

Все девять чисел из таблицы дополнительно проверены независимой реализацией теста простоты при подготовке отчёта. Это проверка результатов конкретного запуска; в самой программе остаётся заданный вариантом тест Рабина—Миллера.

Исходный код расположен в lab1/src/tsevan/main.cpp. Из корня репозитория первая лабораторная собирается и запускается одной командой:

```
.\run.cmd
```

Файлы сборки создаются отдельно от репозитория, в D:/KiSB-extra/build. По умолчанию исследуются 10 000 битов и генерируются три 64-битовых числа. При отдельном запуске lab1.exe из папки сборки аргументы 1000 32 меняют длину выборки и разрядность. Пример вывода сохранён в sample_output.txt рядом с исходником.

Вывод

Реализованы LFSR-генераторы для трёх многочленов варианта 12 и тест Рабина—Миллера. Примитивным оказался $f_1(x) = x^5 + x^2 + 1$, обеспечивающий период 31 при ненулевом начальном состоянии. Эксперимент показал влияние многочлена на период, частоты и автокорреляцию. На основе генератора получены кандидаты заданной разрядности, выполнены просеивание и пять раундов проверки простоты. Короткий период рассматриваемого LFSR ограничивает разнообразие кандидатов и не обеспечивает криптографическую стойкость.